

TAREA INTEGRADORA

UniBot - Sistema Asesor Académico

Algoritmos y Programación I

Nombre: Sebastian Males Salazar

Fecha: 26 de Septiembre de 2026

1. Descripción general del proyecto

UniBot es una aplicación de consola en Node.js pensada para apoyar a estudiantes de primer ingreso. El sistema valida al estudiante mediante su código institucional, identifica su carrera y permite realizar consultas a un asesor virtual. Las respuestas deben limitarse a los temas permitidos para la carrera del estudiante. También existe un módulo de administrador para gestionar accesos, temas permitidos y sesiones.

La solución se plantea de forma incremental. En esta primera entrega se realiza la planeación del sistema desde el enfoque de ingeniería de requisitos. No se desarrolla todavía el código fuente; se define cómo funcionará UniBot, qué necesita, cómo se organizará el trabajo y cuáles serán las tareas iniciales del proyecto.

2. Fase 1 - Pensamiento Sistémico e Ingeniería de Requisitos

2.1 Caracterización del sistema

Objetivo: Brindar orientación académica automatizada a estudiantes de primer ingreso, garantizando que las respuestas se limiten a los temas permitidos para su carrera.

Entradas: Código institucional, contraseña del administrador, preguntas del estudiante, información de estudiantes.csv, información de carreras.json y variables de entorno.

Procesos: Validar usuarios, identificar la carrera, consultar los temas permitidos, realizar consultas al asesor virtual, registrar sesiones, calcular costos, guardar conversaciones y permitir operaciones administrativas.

Salidas: Mensajes de bienvenida, respuestas del bot, mensajes de acceso permitido o denegado, registros de sesión, costo de sesión, historial de conversación y resultados de auditoría.

Subsistemas: Módulo de estudiante, módulo de administrador, autenticación, lectura y escritura de archivos, asesor virtual, gestión de sesiones, auditoría y trazabilidad.

2.2 Modelo de proceso y marco de trabajo

Se propone Scrum porque permite dividir un proyecto grande en entregas pequeñas y revisables. Cada Sprint produce un avance concreto y facilita detectar errores antes de continuar con funciones más complejas.

- **Sprint 1 - Planeación:** Caracterización del sistema, Product Backlog, casos de uso, tablero Kanban y repositorio base.
- **Sprint 2 - Prototipo JavaScript:** Menú dual, contraseña desde .env, lectura de estudiantes.csv y ciclo de validación del estudiante.
- **Sprint 3 - TypeScript y POO:** Migración a TypeScript, creación de clases, gestión de estudiantes y modificación de carreras.json.
- **Sprint 4 - IA y trazabilidad:** PromptBuilder, conexión asíncrona a la API, registro de sesiones, logs, costos, auditoría y manejo de excepciones.

2.3 Product Backlog

	Requerimiento	Tipo	Prioridad
	Mostrar menú para elegir Administrador o Estudiante	Funcional	Alta
	Permitir acceso del estudiante mediante código institucional	Funcional	Alta
	Validar el código utilizando estudiantes.csv	Funcional	Alta
	Permitir acceso del administrador mediante contraseña	Funcional	Alta
	Leer la contraseña del administrador desde .env	No funcional	Alta
	Identificar la carrera del estudiante	Funcional	Alta
	Leer los temas permitidos desde carreras.json	Funcional	Alta
	Permitir al estudiante realizar consultas al chat	Funcional	Alta

	Requerimiento	Tipo	Prioridad
	Restringir las respuestas a los temas permitidos	Funcional	Alta
	Permitir al administrador registrar estudiantes	Funcional	Media
	Permitir agregar temas permitidos a una carrera	Funcional	Media
	Registrar cada sesión de estudiante	Funcional	Media
	Calcular el costo al cerrar la sesión	Funcional	Media
	Guardar el historial de conversación en un archivo .txt	Funcional	Media
	Mostrar sesiones activas y cerradas	Funcional	Media
	Permitir cierre forzado de sesiones activas	Funcional	Media
	Manejar errores de archivos y API con try/catch	No funcional	Alta
	Evitar que un error cierre completamente el programa	No funcional	Alta
	Organizar el proyecto en módulos	No funcional	Media
	Usar TypeScript con strict: true en la versión final	No funcional	Alta

2.4 Historias de usuario principales

- HU01: Como estudiante quiero ingresar con mi código institucional para acceder a UniBot.
- HU02: Como estudiante quiero hacer preguntas relacionadas con mi carrera para recibir orientación académica.
- HU03: Como estudiante quiero recibir respuestas únicamente de los temas autorizados para evitar información de otra carrera.
- HU04: Como administrador quiero autenticarme con una contraseña para acceder al Back-Office.
- HU05: Como administrador quiero registrar nuevos estudiantes para habilitar su acceso.
- HU06: Como administrador quiero agregar temas permitidos para mantener actualizado el contexto de una carrera.
- HU07: Como administrador quiero auditar las sesiones para conocer cuántas están activas o cerradas.
- HU08: Como sistema quiero guardar el historial de cada chat para conservar trazabilidad.

2.5 Diagrama UML de casos de uso

El diagrama de casos de uso se entrega como un archivo independiente. Allí se muestran las interacciones principales de los dos actores del sistema: Estudiante y Administrador.

2.6 Tablero Kanban

El tablero Kanban se entrega como un archivo independiente. Inicialmente, las tareas del Product Backlog se ubican en la columna Por hacer y durante el desarrollo se moverán a En proceso y Terminado.

3. Reflexión sobre el uso de IAG

Durante esta primera entrega se utilizó una herramienta de inteligencia artificial como apoyo para comprender el enunciado, organizar los requerimientos y revisar el Product Backlog y las historias de usuario. Las respuestas se compararon con la consigna y se adaptaron a los conceptos trabajados en clase. La herramienta ayudó a dividir el problema en tareas pequeñas, pero fue necesario revisar el resultado antes de usarlo.

Ejemplos de prompts utilizados:

- Ayúdame a separar los requerimientos de UniBot en funcionales y no funcionales.
- Explícame la caracterización del sistema con objetivo, entradas, procesos, salidas y subsistemas.
- Ayúdame a organizar el Product Backlog de la primera fase.